Find Hardcoded Credentials – Complete Guide

Hardcoded credentials ka matlab hai ke API keys, tokens, passwords, aur doosre sensitive credentials directly source code mein likhe gaye hain . Android APK mein yeh sab se common vulnerability hai kyunke APK decompile karke koi bhi easily in secrets ko access kar sakta hai . OWASP Mobile Top 10 mein bhi insecure data storage aur hardcoded secrets ko important vulnerabilities mana gaya hai .

Hardcoded Credentials Kahan Milte Hain?

strings.xml – Developer aksar credentials strings.xml mein store karte hain . Facebook App ID, Facebook Client Token, Google API Key, aur Firebase keys aam taur par yahan se mil jaate hain . Jaise ek real example mein strings.xml se Facebook App ID, Google API Key, aur Firebase storage bucket mil gaye the .

AndroidManifest.xml – Ismein bhi kuch API keys ya meta-data ke taur par sensitive information store hoti hai .

Smali code – Decompiled smali files mein bhi hardcoded strings aam taur par milte hain . Strings search karke aap easily credentials find kar sakte hain .

assets/ folder – Kuch apps assets folder mein configuration files, JSON files, ya plain-text files store karti hain jinmein credentials ho sakte hain . Jaise example mein `assets/sensitive.txt` file plain-text secret store kar rahi thi .

Native libraries (.so files) – C/C++ code mein bhi hardcoded strings mil sakti hain . Key Hunter tool native libraries ko bhi scan karta hai aur `.so` / `.a` files se ASCII aur UTF-16LE strings extract karta hai .

Manual Tareeqa – Kahan Dekhein

Step 1: APK decompile karein

```
apktool d target.apk -o output_folder -f
```

Step 2: strings.xml check karein

`res/values/strings.xml` mein suspicious keys search karein jaise "api_key", "secret", "token", "password", "client_id" .

Step 3: grep use karein

```
grep -r -i "AIza" output_folder/
grep -r -i "facebook_client_token" output_folder/
grep -r -i "api_key" output_folder/
```

Step 4: Java/Smali code check karein

JADX-GUI open karein aur Ctrl+Shift+F se keywords search karein: "password", "token", "secret", "key", "AES", "RSA" .

Automated Tools – Secret Finder

Secret Finder Python-based command-line tool hai jo 40+ regex patterns use karta hai APK mein hardcoded secrets detect karne ke liye . Yeh high-entropy strings, specific key patterns, aur doosre sensitive data ko identify karta hai . Multi-processing use karta hai taake scan fast ho, aur professional HTML report generate karta hai jismein severity ranking (Critical/High/Medium/Low) aur code context preview hota hai .

Installation:

```
git clone https://github.com/viralvaghela/secret-finder.git
cd secret-finder
pip install -r requirements.txt
```

Usage:

```
python secret_finder.py
```

Scan options:

- Basic scan – Fast, sirf AndroidManifest.xml aur strings.xml check karta hai

- Advanced scan – Poori APK decompile karke har file scan karta hai

Output: `security_report_<apk_name>.html` generate hoti hai jo interactive dashboard ke saath aati hai .

Automated Tools – ohmyg0sh

ohmyg0sh Dart-based APK security scanner hai jo jadx ke zariye APK decompile karta hai aur 50+ built-in regex patterns apply karta hai . Cloud services (AWS, Google Cloud, Azure, DigitalOcean), social media (Facebook, Twitter, Slack, Discord), payment (Stripe, PayPal), aur developer services (GitHub, GitLab) ke liye patterns included hain .

Installation:

```
dart pub global activate ohmyg0sh
```

Usage:

```
ohmyg0sh -f app.apk
ohmyg0sh -f app.apk --json -o results.json
```

Custom patterns:

```
ohmyg0sh -f app.apk -p custom-patterns.json
```

Yeh configurable detection rules, false-positive filters, JSON aur text output formats provide karti hai .

Automated Tools – Key Hunter

Key Hunter sab se advanced tool hai jo 60+ detection rules aur severity-based classification provide karta hai . Yeh Retrofit aur OkHttp endpoints extract karke Postman collection bhi generate kar sakta hai . FP filtering aur third-party SDK path exclusion ke zariye false positives ko filter karta hai .

Key Features:

- 60+ named rules – AWS, GCP, Azure, Stripe, Slack, Discord, Telegram, Twilio, GitHub, Firebase, PEM/PGP/PuTTY keys, JWT, Mongo/Postgres/MySQL URIs, etc.
- Severity: Critical, High, Medium, Low, Info
- Native library support – `.so` aur `.a` files scan karta hai
- Multiple output formats – txt, json, html, sarif
- HTML report – tabbed layout, search, severity filter
- SARIF support – GitHub code-scanning integration

Installation:

```
git clone https://github.com/0xj3st3r/key-hunter.git
cd key-hunter
pip install -r requirements.txt
```

Usage:

```
key-hunter --apk app.apk --html
key-hunter --apk app.apk --api --include-native
```

Verification – Credentials Valid Hain Ya Nahi

Ek real example mein strings.xml se Facebook App ID aur Client Token mil gaye . Unko verify karne ke liye Graph API call ki gayi:

```
curl
"https://graph.facebook.com/app?access_token=APP_ID|CLIENT_TOKEN"
```

Response se confirm hua ke credentials valid hain . Is tarah attackers app impersonate kar ke Graph API calls bhej sakte hain .

Google API key bhi valid thi, lekin developer ne Google Cloud Console mein access restrictions set kar rakhi thin (package name aur SHA-1 fingerprint based), isliye misuse nahi ho paayi .

Manual Keyword Search – Best Practices

Common strings search karein:

- `AIza` – Google API key prefix

- `AKIA` – AWS access key prefix
- `facebook_client_token` – Facebook token
- `sk_live` / `pk_live` – Stripe live keys
- `xoxb-` / `xoxp-` – Slack tokens
- `ghp_` – GitHub personal access token
- `SG.` – SendGrid API key
- http:// / https:// – Hardcoded URLs

Best Practices for Developers

- **Never store secrets in strings.xml** – Strings.xml compile hoti hai aur easily accessible hoti hai . Agar credentials client-side chahiye toh NDK use karke native code mein store karein aur Android Keystore use karein . Ya credentials server se dynamic fetch karein .
- **Google API keys par restrictions apply karein** – Package name aur SHA-1 fingerprint-based restrictions lagayein . Sirf required APIs enable karein .
- **ProGuard/R8 use karein** – Code obfuscate karein (rename classes/methods) taake reverse engineering thoda mushkil ho . Shrinking aur optimization bhi enable karein .
- **Runtime signature check implement karein** – APK tampered hai ya nahi check karein . Debug ke liye logs strip karein aur release builds mein debug info remove karein . Play Integrity API bhi ek additional security signal hai .

📘 **Educational Purpose Only:** Yeh guide sirf educational aur learning purposes ke liye hai. Hardcoded credentials ka discovery sirf apne own apps aur authorized bug bounty programs mein karein. Kisi doosre ke app mein bina permission ke credentials find karna aur exploit karna applicable cyber laws ke under punishable hai.

MADE BY:
SYED GHOUS
